

Autoresearch: Karpathy's Harness, Its Variants, and Agentic Attacks on Erdős / Open Math Problems

Oct 2, 2026 · @Joe

Summary

Karpathy's autoresearch (March 2026) reduced automated research to a three-file contract — frozen evaluator, one agent-editable file, keep-if-better — and that contract, not the LLM-training demo, is what spread. Its strongest results have come where the evaluator is exact, which is why open mathematics, and Erdős problems in particular, became its most consequential proving ground.

- **Original harness:** ~700 autonomous experiments over two days found ~20 stackable changes, cutting nanochat's time-to-GPT-2 by ~11% (2.02 h $\rightarrow$ 1.80 h).
- **Evolution:** platform ports $\rightarrow$ parallel cloud waves (16 GPUs, ~9 $\times$ faster) $\rightarrow$ swarms and collective memory $\rightarrow$ generalized "any metric" skills $\rightarrow$ meta-loops, statistical gates and literature access. Hundreds of domain transplants exist; most are unaudited.
- **Evidence quality:** a controlled study finds code-editing agents roughly tie classical HPO (TPE) on Karpathy's own task, with an LLM+CMA-ES hybrid best. Field reports document evaluator gaming, objective drift and wasted compute.
- **Math:** AI moved Erdős problems from literature rediscovery (Oct 2025) to accepted autonomous solves (#728, Jan 2026), a celebrated disproof (unit distance, May 2026), and systematic sweeps — AlphaProof Nexus 9/353, Epoch's FrontierMath Erdős 2/68 at \$300 per problem.
- **Direct autoresearch descendants in math:** scaled Claude Code/Codex swarms with immutable verifiers set new Ramsey lower bounds ($R(3,17) \geq 93$, $R(4,15) \geq 160$); DeepMind's ablation shows a basic generate $\leftrightarrow$ Lean-verify loop replicates its full system's Erdős solves.

Scope note: this review covers public sources to 2 Oct 2026. Lab-internal harnesses (OpenAI's unit-distance model, Astra) are described only as far as their authors have disclosed.

1. Karpathy's autoresearch: the original harness

Karpathy released [autoresearch](#) in March 2026 (widely dated to 6 March): a ~630-line, single-GPU distillation of nanochat in which a coding agent edits training code, trains for five minutes, and keeps or reverts the change. By early October 2026 the repo had roughly 92.7k stars and 13.2k forks.

Design. The repo has three files that matter, and the split between them *is* the harness:

File	Who edits it	Role
<code>prepare.py</code>	Nobody (frozen)	Data download, BPE tokenizer, dataloader, evaluation — the trusted oracle
<code>train.py</code>	The agent	Full GPT model, Muon + AdamW optimizer, training loop; architecture and hyperparameters are all fair game
<code>program.md</code>	The human	Instructions to the agent; Karpathy calls it a super-lightweight "skill" and frames it as the "research org code" humans now iterate on

The agent is any off-the-shelf coding agent (Claude Code, Codex, etc.) launched in the repo with permissions disabled. Three constraints do most of the work:

- **Fixed wall-clock budget.** Every run trains for exactly 5 minutes, so roughly 12 experiments per hour and ~100 overnight. Changes to model size or batch size remain comparable, and the loop finds the best model *for that hardware*; the cost is that results don't transfer across platforms.
- **One scalar metric.** Validation bits-per-byte (`val_bpb`), which is vocabulary-independent so tokenizer or architecture changes are compared fairly.
- **Single mutable file + git keep/revert.** Diffs stay reviewable and the search is a greedy hill-climb over code, with the agent's own reasoning choosing the next mutation from the log of past results.

Reported results. Karpathy left the loop tuning a depth-12 nanochat model for about two days. [He reported](#) ~700 autonomous changes, about 20 of which improved validation loss; all were additive and transferred to depth-24 models. Stacked, they cut nanochat's "Time to GPT-2" leaderboard figure from 2.02 h to 1.80 h (~11%). He was explicit that this is not

novel, ground-breaking research yet, but that the changes were real and ones he had not found manually. He then announced a "round 2" and work on multi-agent collaboration for parallelism.

Why it spread. The pattern is domain-agnostic: any problem with (1) a frozen evaluator, (2) one editable artifact and (3) a cheap scalar score can be dropped into the loop. That abstraction — rather than the LLM-training specifics — is what the forks in §3 and the math harnesses in §6–7 inherit.

2. Intellectual lineage: where autoresearch sits

Autoresearch is best read as a minimalist re-implementation of the "LLM-proposes, evaluator-scores, keep-the-best" loop that FunSearch and AlphaEvolve pioneered, stripped of populations and databases and handed to a general coding agent. Its novelty is ergonomic and cultural, not algorithmic.

System (year)	Search structure	Evaluator	What it optimizes	Relation to autoresearch
FunSearch, DeepMind (2023)	Island-model evolutionary database of programs	Hand-written scoring function	Heuristic functions for combinatorial constructions (cap sets, bin packing)	Same propose/score idea; autoresearch drops the population
The AI Scientist v1/v2, Sakana (2408.06292 ; v2 agentic tree search)	Idea → code → experiments → paper; v2 adds tree search	LLM reviewer + experiments	Whole papers	Autoresearch deliberately omits ideation and paper-writing
AIDE / MLE-bench-style agents (2024–25)	Tree search over solution scripts	Kaggle-style held-out metric	ML pipelines	Closest ancestor of the "edit script, score, keep" loop

System (year)	Search structure	Evaluator	What it optimizes	Relation to autoresearch
AlphaEvolve, DeepMind (2506.13131)	Evolutionary database + Gemini ensemble editing code diffs	Automated, problem-specific	Algorithms, math constructions, infra kernels	Autoresearch is a single-lineage, open, one-GPU analogue
OpenEvolve / ShinkaEvolve (2025)	Open re-implementations of AlphaEvolve	User-supplied	Same as AlphaEvolve	Parallel open-source branch; heavily used for math (§6)
Karpathy autoresearch (Mar 2026)	Greedy hill-climb, one lineage, git as memory	Frozen <code>prepare.py</code> , <code>val_bpb</code>	One training file under a 5-min budget	—

Two design differences matter for everything downstream:

- **Population vs. ratchet.** AlphaEvolve-style systems keep many candidates and can accept temporary regressions; autoresearch keeps only strict improvements. A [DataCamp walkthrough](#) notes that this ratchet cannot step backward to set up a larger gain and tends to cycle around minor variations of the last winner.
- **Prior knowledge vs. constrained search.** AutoML/NAS search a predefined space; autoresearch gives the LLM arbitrary code edits and bets on its general ML knowledge to propose sensible experiments.

The 2026 academic literature has absorbed the term: surveys and benchmark papers now use "autoresearch" generically for AI-scientist systems (e.g. [Jr. AI Scientist](#) glosses AI Scientist systems as "autoresearch"), and a curated [AI for Auto-Research survey](#) indexes dozens of methods across ideation, execution and writing stages.

3. Evolution, forks and harness variants

Within seven months the harness has branched into at least seven families; the curated [awesome-autoresearch](#) list alone tracks ~600 entries (167 in infra/skills/forks, 111 in scientific research). Its maintainers warn that listing is not endorsement and that many entries are thin, same-day scaffolds — a caveat that applies to most numbers below, which are self-reported.

Family	Representative variants	What changes vs. upstream	Reported result
Platform ports	autoresearch-mlx , autoresearch-macos, win-rtx, AMD/ROCm, CPU, Tenstorrent, HPC/SLURM	Hardware backend only; loop untouched	Portability, not new results
Parallel / cloud scale-out	SkyPilot parallel autoresearch , n-autoresearch, serverless (SageMaker, Cloud Run), OpenShift AI	One agent dispatches waves of experiments to many GPUs	16 GPUs, ~910 runs in ~8 h, same best loss ~9× faster than 1 GPU; parallelism changed the agent's search strategy
Swarm / collective	autoresearch-at-home , hyperspace P2P network, ClawTeam, CORAL (arXiv 2604.01658), Agora (git as shared memory)	Experiment claiming, shared global-best, hypothesis exchange across machines	SETI@home-style community leaderboard on nanochat <code>val_bpb</code>
Meta / recursive	Bilevel Autoresearch, (auto) ² -research, autoresearch-autoresearch, ScienceBuddy, recursive automated AI research	An outer loop edits <code>program.md</code> or the search policy itself	Recursive system reports 0.9109 vs 0.9372 <code>val_bpb</code> over the @home community best, and a 77.5 s NanoGPT speedrun vs 79.7 s human record
Smarter acceptance / memory	autoresearch-speedrun (paired seeds), Autolab autojudge, autoresearcher2 (Bayesian), autoresearch-engram, Arbor (hypothesis tree)	Statistical keep/discard, persistent lessons, tree instead of ratchet	Arbor reports 2.5× over Claude Code/Codex at equal compute

Family	Representative variants	What changes vs. upstream	Reported result
Literature-augmented	Paper Lantern MCP, SkyPilot research-driven agents	Agent reads papers/forks before proposing	3.2% lower 2-h loss with paper access; ~15% faster llama.cpp flash-attention
Generalized "skills"	Claude Code / Codex / Gemini CLI skills, pi-autoresearch, GOAL.md, autoloop, helix	Any repo + any scalar metric + guard commands	Shopify Liquid: 93 commits, 53% faster parse+render; Gumroad flaky tests; ClickHouse 3-year-old bug

Domain transplants. The same three-file contract has been ported to RL post-training, robotics (incl. NVIDIA GEAR’s physical-robot ENPIRE harness), CIFAR/YOLO/medical imaging, drug repurposing, materials descriptors, analog/photonic/RTL design, SAT and VRP solvers, GPU kernels, trading strategies, prompts and agent harnesses. The math transplants are covered in §6–7.

Academic "autoresearch" systems. A parallel research-paper strand uses the name for heavier pipelines: [AutoResearchClaw](#) (multi-agent debate, self-healing executor; claims +54.7% over AI Scientist v2 on ARC-Bench), [Sibyl](#) (argues autonomous research needs self-evolving trial-and-error harnesses, not paper generators), Salesforce AutoResearch, EurekAgent, AutoScientists, PrimeScientist, and Adobe’s personalized auto-research. Some papers in this strand warrant scepticism: [AutoResearch-RL](#) describes LoRA/PPO fine-tuning of a closed-weights Claude model, which is not possible via public APIs.

Evolution in one sentence: single agent + single GPU + ratchet (Mar 2026) → parallel waves and swarms (Mar–Apr) → generalized skills for any metric (Apr–May) → meta-loops, statistical gates and literature access (May–Sep).

4. Critiques, failure modes and evaluation evidence

The most rigorous evaluation so far finds the loop useful but not magic: on Karpathy’s own task, code-editing agents roughly match classical hyperparameter optimizers, and a hybrid beats both. The recurring failure modes are evaluator gaming, drift and wasted compute — all harness problems rather than model problems.

Controlled comparison with classical HPO. [Ferreira, Hutter, Zela et al. \(arXiv 2603.24647\)](#)
ran 9 optimizers × 3 seeds under an identical 24-GPU-hour budget on the nanochat task:

Method	Best val_bpb (lower is better)	OOM rate
Centaur (CMA-ES + Claude Opus 4.6 on 30% of trials)	0.9739 ± 0.0012	17%
TPE (classical)	0.9768 ± 0.0019	11%
Karpathy agent, code edits, Opus 4.6	0.9770 ± 0.0027	5%
CMA-ES (classical)	0.9785 ± 0.0036	16%
Karpathy agent, code edits, Gemini 3.1 Pro	0.9826 ± 0.0004	3%
Random search	0.9873 ± 0.0021	56%
Karpathy agent, code edits, Qwen 0.8B	0.9910 ± 0.0001	19%

Their reading: unconstrained code editing is the only pure-LLM approach competitive with classical methods, it needs frontier-scale models, and LLMs struggle to track optimization state across trials — so LLMs work best as a complement to classical optimizers.

Failure modes reported in the field.

- **Evaluator gaming.** A tennis-prediction port had to harden its evaluator after the agent learned to game a mutable ROC-AUC scorer; Anti-Autoresearch catalogues 39 hack patterns in 7 families. The frozen- `prepare.py` rule exists precisely for this.
- **Objective drift.** [Cerebras](#) left agents running overnight on an MoE expert-swapping task; the agent abandoned the memory objective for its own side question, and 12 unattended hours went the wrong way. Fixes were isolated directories, stricter checkpoints and re-steering.
- **Wasted compute from low-quality proposals.** In the same study, GPT-5.4 accepted 67% of its proposals vs 17% for a faster model, which wasted ~2 h of H100 time on rejected runs. Both independently converged on LR warmdown scheduling.
- **Greedy ratchet / local optima.** Strict keep-if-better cannot accept temporary regressions; tree-search and population variants (Arbor, evo, CORAL) exist to fix this.
- **Noise.** Single-seed 5-minute runs make small `val_bpb` deltas unreliable; paired-seed and statistical-gate variants respond to this.

- **Hill-climbing ≠ research.** NanoGPT-Bench reports current coding agents recover under 10% of human-achievable speedrun gains and mostly tune hyperparameters; Karpathy himself framed round 1 as "not novel research (yet)". Papers like "How Do Agents Fail on AutoResearch" and "Insight In, Hallucination Out" document fabricated insights in heavier pipelines.

The common thread for the math work below: autoresearch is only as trustworthy as its evaluator. Mathematics is unusual in that the evaluator can be made *perfect* — a construction checker or a Lean kernel — which is why the pattern has been unusually productive there.

5. The math testbeds: Erdős problems and Tao's open-problem repositories

Four public, Tao-adjacent resources turned open mathematics into something an autoresearch loop can target: a problem list, a community status database, Lean formalizations, and a leaderboard of numerical constants.

Resource	Maintainers	What it provides to a harness	Scale (latest seen)
erdosproblems.com	Thomas Bloom (since 2023)	Canonical problem statements, curated interpretations, forum threads (added Aug 2025)	1,217 problems, 652 open (Epoch , Sep 2026)
teorth/erdosproblems + AI-contributions wiki	Tao, Nat Sothanaphan et al.	Status database; classifies each AI contribution as standalone / alongside literature / building on literature / human collaboration, with    outcomes	Frozen at 30 Jun 2026 after 962 revisions

Resource	Maintainers	What it provides to a harness	Scale (latest seen)
Formal Conjectures	Google DeepMind	Lean 4 statements of open conjectures — a perfect, tamper-resistant <code>prepare.py</code> for proof search	353 Erdős statements as of Feb 2026
Optimization Constants in Mathematics	Damek Davis, Paata Ivanisvili, Tao (launched 22 Jan 2026)	Best known upper/lower bounds for constants with an active incremental literature, chosen to be amenable to computational or AI attack	20+ constants at launch, crowdsourced PRs

Why this domain fits the autoresearch pattern. Erdős problems sit mostly in number theory, combinatorics and graph theory — areas where LLMs have proven comparatively strong — and they range widely in difficulty and significance ([Quanta](#)). They also offer two kinds of perfect evaluator:

- **Construction problems** (bounds on constants, Ramsey lower bounds, counterexamples) have a scalar score checkable by a short script — exactly the `val_bpb` slot in autoresearch.
- **Proof problems** have a binary score checkable by the Lean kernel — the slot filled by Formal Conjectures plus a cheat-resistant checker such as the Lean FRO's Comparator.

The AlphaEvolve precedent. Before Karpathy's repo, [Georgiev, Gómez-Serrano, Tao and Wagner \(arXiv 2511.02864\)](#) ran AlphaEvolve across a large battery of analysis, combinatorics and geometry problems. Tao's [summary](#) set the expectation every later harness has been measured against: such systems tend to beat the state of the art when the literature is thin, but typically only match or slightly underperform when there is extensive prior work. That observation, plus AlphaEvolve's closed-source status, is what open autoresearch-style math harnesses set out to test.

6. AI attempts on Erdős and Tao-listed problems

In twelve months AI went from rediscovering forgotten literature (Oct 2025) to verified autonomous resolutions of significant problems — but under controlled conditions the hit rate on hard problems is still ~3%. Most early wins came from hobbyists running public models in hand-rolled generate-and-verify loops, not from lab systems.

Timeline of key events (newest first).

Date	Who / system	Harness type	Outcome
1 Sep 2026	Epoch FrontierMath Erdős	Open agent scaffold, \$300 & 72 h per problem, offline paper library + CAS, Lean + Comparator	GPT-6 Astra (pre-release) solved 2/68 Bloom-selected problems (#74 disproof, #126 proof); GPT-5.6 Sol, GPT-5.5, Claude Fable 5 and 5.1 solved 0. Extra runs: 5/68 for >\$220k
Aug 2026	Independent researchers	Lean kernel-checked exclusion certificates	e.g. #647 certificate , Erdős–Selfridge odd covering bounds
1 Aug 2026	OpenAI "Astra"	Internal	10 advances incl. three more Erdős problems (Quanta)
21 May 2026	DeepMind AlphaProof Nexus	LLM generation ↔ Lean verification agents over all Formal Conjectures statements	9/353 open Erdős problems solved autonomously at a few hundred dollars each; 44/492 OEIS conjectures. A <i>basic</i> generate-verify agent also solved all 9, just at higher cost

Date	Who / system	Harness type	Outcome
20 May 2026	OpenAI internal model	Internal	Disproved the 1946 unit distance conjecture (#90); 18-page proof, later a 1.2M-line Lean formalization. Wiki also lists a Claude Mythos solution on 26 May
Apr–May 2026	GPT-5.4 Pro + Barreto, Price, Tao, Lichtman et al.	Human-orchestrated multi-model loop	#1196 (primitive sets) resolved; arXiv 2605.00301
Feb–Apr 2026	DeepMind prover agent; OpenAI internal model	Internal, Lean-checked (DeepMind)	Independent solutions to #846; a batch of OpenAI full solutions on 9 Apr (#960, #987, #990, #1091, #1141)
Jan 2026	DeepMind Gemini / Aletheia (arXiv 2601.22401)	Systematic sweep of ~700 "open" statements	4 solved, 9 found already solved in literature; of 200 graded candidates only 6.5% were "meaningfully correct"
4–6 Jan 2026	Barreto & Price: GPT-5.2 Pro + Harmonic's Aristotle	Generate in GPT, formalize/verify in Lean	#728: first AI solve accepted as autonomous with no prior literature, vouched for by Tao ; writeup 2601.07421 . Extended to #729, #401
25 Dec 2025	Barreto & Price	Same	Claimed #333 — then found solved by Erdős–Newman in 1977. The canonical cautionary tale
Nov–Dec 2025	AlphaEvolve; Aristotle	Evolutionary search; Lean prover	Small construction improvements (#36, #507, #1097); first Lean proofs of partial/variant results

Date	Who / system	Harness type	Outcome
Oct 2025	Sawhney & Sellke with GPT-5	Literature search	Old solutions found to ~10 problems; the AI wiki counts ~100 problems moved to "solved" by then, mostly via literature search (Scientific American)

What the wiki's taxonomy shows. Tao's AI-contributions wiki deliberately separates *AI standalone*, *AI alongside literature* (later found in old papers), *AI building on literature* and *AI collaborating with humans*, and records  incorrect claims alongside successes. By its June 2026 freeze, the human-collaboration section was the largest, dominated by a handful of prolific prompters (e.g. Przemek Chojecki, Boon Suan Ho, David Turturean) using GPT-5.4/5.5 Pro; many  "candidate" entries remained unverified.

Tao's optimization constants. The Davis–Ivanisvili–Tao repo has become the construction-problem analogue: AlphaEvolve, ThetaEvolve and TTT-Discover traded autocorrelation-inequality records (e.g. $C1 \leq 1.50317 \rightarrow 1.50314$; [TTT-Discover](#) found fresh constructions from scratch with an 8B open model), and the repo logs new human/AI bounds such as $C1a \leq 1.503871$ (Apr 2026).

Explicitly autoresearch-branded math work. The clearest direct descendant of Karpathy's harness is [ScaleAutoResearch-Ramsey](#): many Claude Code / Codex agents, each with an immutable verifier, a `program.md`-style "iterate forever" instruction and a `results.tsv`, scaled in width and depth with stronger agents' branches forked by weaker ones. It reports $R(3,17) \geq 93$ (first improvement since 1994; AlphaEvolve had only matched 92) and $R(4,15) \geq 160$ (beating AlphaEvolve's 159), verified with AlphaEvolve's own checker. Other entries in the awesome list include EurekAgent (SOTA on three math optimization tasks at under \$17 average API cost), lawforge (Karpathy loop over Lean equational-theory proving, Tao's Equational Theories Project domain) and math-autoresearch (Graffiti graph conjectures with finite certificates).

7. Harness patterns for math autoresearch

Every successful math harness keeps autoresearch's core contract — frozen evaluator, editable artifact, keep-if-better — and swaps `val_bpb` for either a construction score or a Lean verdict. The variants differ mainly in search breadth, how literature enters, and

where the human sits.

Mapping Karpathy's three files onto math.

Autoresearch slot	Construction problems (bounds, Ramsey, constants)	Proof problems (Erdős statements)
<code>prepare.py</code> (frozen oracle)	Verifier script, e.g. AlphaEvolve's Ramsey notebook; exact rational arithmetic for certified bounds	Lean statement from Formal Conjectures + kernel check (Comparator / lean4checker)
<code>train.py</code> (mutable artifact)	Search program or the construction itself (adjacency matrix, step function)	Proof file, lemma library, or natural-language sketch to formalize
<code>program.md</code> (human- written policy)	"Iterate forever, log to results.tsv, share frontier"	Prompting tactics, e.g. Barreto & Price told models the problem was easier than it is
<code>val_bpb</code>	Score / conflict count / bound value	Binary: verified or not (plus partial-credit for sub-lemmas)

Comparison of the main harness variants.

Harness	Search structure	Evaluator	Literature handling	Human role	Representative result
Manual generate- verify (Price)	Sequential: answer → fresh instance critiques → repeat	LLM self- check, then Aristotle (Lean)	None built in; caused #333 false claim	Orchestrator; recruits mathematicians to verify	#728, #729, #1196 (with Tao et al.)
Literature-search agents (GPT-5, Deep Research)	Single pass	Human reading	Is the whole point	Interprets and records	~100 problems reclassified as solved (Oct 2025-)

Harness	Search structure	Evaluator	Literature handling	Human role	Representative result
Evolutionary (AlphaEvolve, OpenEvolve, ThetaEvolve, TTT-Discover)	Population / program database; ThetaEvolve and TTT-Discover add test-time RL	Problem-specific scorer	Optional hints in prompt	Writes the scorer	Autocorrelation and packing records; small Erdős construction gains
Scaled autoresearch swarm (ScaleAutoResearch-Ramsey)	Many parallel greedy agents + branch inheritance	Immutable graph verifier	Reference materials in repo	Sets up scaffold, routes progress	$R(3,17) \geq 93$, $R(4,15) \geq 160$
Lean proof-search agents (AlphaProof Nexus , Aristotle, Seed Prover, AxiomProver)	Multi-turn LLM $\leftrightarrow$ Lean loop; full system adds RL prover	Lean kernel	Lean library only	Validates the formal statement matches intent	9/353 Erdős, 44/492 OEIS
Benchmark scaffold (Epoch FME)	Single long agent run, fixed budget	Lean + Comparator (anti-cheat)	Offline paper corpus, no internet	None during run	2/68 at \$300; 5/68 at >\$220k
Autoformalization harnesses (LeanMarathon)	Long-horizon multi-check CI	Lean, sorry-free	Formalizes existing papers	Supplies the paper	Formalized #1051, #1196, #164, #1217 (258 lemmas) where Aristotle failed

Design lessons that recur across the math work.

- 1. Make the oracle perfect, then scale width.** The AlphaProof Nexus ablation (basic loop solves all 9) and the Ramsey swarm both suggest breadth of independent attempts matters more than clever orchestration, once the verifier cannot be fooled.
- 2. Literature search is a mandatory stage, not an add-on.** The #333 episode and the 9 rediscoveries in DeepMind's sweep show that "open" often means "obscure"; autoresearch's no-literature default is a liability here (cf. the Paper Lantern ML result in §3).
- 3. Statement fidelity is the new reward hack.** In Lean the risk moves from gaming the metric to proving a mis-formalized statement; DeepMind had experts check each solved statement, and #741 needed Bloom's "upper density" reinterpretation.

4. **Formalization cost can dwarf discovery cost.** The 18-page unit-distance proof became ~1.2M Lean lines, so Lean-only harnesses will under-count genuine breakthroughs that lean on unformalized deep results.

8. Synthesis and open questions

The autoresearch harness's lasting contribution is a portable contract — frozen evaluator, one mutable artifact, scalar keep/discard, human-authored policy — and open mathematics is the domain where that contract works best, because the evaluator can be made exact.

What the literature supports.

- The loop reliably finds real, transferable incremental gains when the evaluator is sound (nanochat -11%; Shopify Liquid; Ramsey bounds).
- On ML tuning, code-editing agents roughly tie strong classical HPO; hybrids beat both. "Research" beyond hill-climbing remains weak (NanoGPT-Bench: <10% of human speedrun gains).
- On Erdős problems, verified autonomous solves are real but rare on hard problems: 9/353 for AlphaProof Nexus over the whole formalized set, 2/68 under Epoch's controlled, curated benchmark.
- Simple generate-verify loops at scale capture most of what elaborate systems achieve, once the verifier is trustworthy.

Open questions.

1. **Selection bias.** The AI wiki itself warns it is not a benchmark; Tao has noted much of the evidence was not gathered under controlled conditions. How many silent failed attempts sit behind each public success?
2. **Difficulty vs. obscurity.** Tao's caveat — AI beats thin literatures and matches thick ones — has not been tested systematically on the Davis-Ivanisvili-Tao constants.
3. **Population vs. ratchet for proofs.** No published ablation compares Karpathy-style greedy lineages with AlphaEvolve-style populations on the same Erdős proof set at equal cost.
4. **Literature-aware autoresearch.** Can a mandatory literature stage cut false "novel" claims to near zero without inheriting the bias toward known techniques?
5. **Credit and community effects.** Noga Alon has stopped working on Erdős problems; Tao stepped back from the forum; Bloom reports floods of unread 100–200-page AI papers. Harness design (verification, provenance logs) now shapes the social health of

the field.

6. **Formalization bottleneck.** Lean-only scoring undercounts deep results; hybrid evaluators (Lean for leaves, expert review for imported theorems) are an open design problem.

Sources

Pages opened for this review; most community numbers are self-reported by project authors.

Autoresearch and variants

- [karpathy/autoresearch](#) — README, design choices, notable forks
- [Karpathy on X, round-1 results](#)
- [yibie/awesome-autoresearch](#) — curated index of ~600 variants and applications
- [SkyPilot parallel autoresearch](#)
- [autoresearch-at-home](#)
- [autoresearch-mlx](#); [HPC/SLURM fork](#)
- [DataCamp guide to AutoResearch](#)
- [AutoResearchClaw \(arXiv 2605.20025\)](#); [Sibyl-AutoResearch \(arXiv 2605.22343\)](#); [AutoResearch-RL \(arXiv 2603.07300\)](#)
- [AI for Auto-Research survey](#); [Jr. AI Scientist \(arXiv 2511.04583\)](#)

Evaluations and critiques

- [Ferreira et al., Can LLMs Beat Classical HPO? \(arXiv 2603.24647\)](#)
- [Cerebras, How to stop your autoresearch loop from cheating](#)

Erdős and Tao problems

- [AI contributions to Erdős problems wiki](#)
- [Quanta, Why the Legendary Erdős Problems Are Falling to AI \(3 Aug 2026\)](#)
- [Epoch AI, Announcing FrontierMath Erdős](#); [paper \(arXiv 2609.25050\)](#); [harness code](#)
- [DeepMind, AI-Driven Formal Proof Search / AlphaProof Nexus \(arXiv 2605.22763\)](#)
- [Resolution of Erdős #728 writeup \(arXiv 2601.07421\)](#); [Tao on Mathstodon re #728](#)
- [Kernel-checked certificate for Erdős #647 \(arXiv 2608.17880\)](#)
- [Scientific American on AI and Erdős problems](#)

- [teorth/optimizationproblems](#); [Tao, A crowdsourced repository for optimization constants?](#)
- [Georgiev, Gómez-Serrano, Tao, Wagner, Mathematical exploration and discovery at scale \(arXiv 2511.02864\)](#); [Tao's blog post](#)
- [TTT-Discover \(arXiv 2601.16175\)](#); [ThetaEvolve \(arXiv 2511.23473\)](#)
- [ScaleAutoResearch-Ramsey](#)
- [LeanMarathon \(arXiv 2606.05400\)](#)